TABLE OF CONTENT

PIMIZ

Railway management system project	**TEAM:** Piyal & Mirza

NORTHMOST PRESS

1. Acknowledgment

The completion of this **Railway Management System (RMS)** project brings a sense of satisfaction and achievement. This project would not have been possible without the support and guidance of several individuals and resources.

First and foremost, I would like to express my sincere gratitude to my computer science teacher, for providing the opportunity to work on this project and for their invaluable guidance throughout the development process. Their insights into database management and Python programming were instrumental in shaping the logic of this system.

I am also thankful to my institution, **National High School**, for providing the facilities and the environment necessary to conduct this technical study and implementation. I am also thankful to our computer science teacher Ms. Riya Sarkar.

A special note of thanks goes to the developers of the **Python** programming language and the **MySQL** community. The robust libraries such as `mysql-connector-python` and the `tkinter` toolkit provided the essential building blocks for creating a seamless bridge between a user-friendly interface and a complex relational database.

I want to thank my family and friends for their continuous encouragement and for providing the support system required to see this project through from the initial requirement analysis to the final testing phase.

Lastly, I would like to thank myself, for being resillient about the process.

2. Introduction

Problem Statement

Traditional manual railway management systems often suffer from significant drawbacks, including data redundancy, human error in record-keeping, and slow processing times for critical tasks like ticket booking and cancellation. Based on the project implementation, several specific challenges are addressed:

- **Manual Fare and Refund Calculation**: Manual calculation of refunds is prone to error; the code automates this by calculating a 80% refund of the original fare upon cancellation.
- **Data Fragmentation**: Information regarding trains, passengers, and bookings is often stored in disparate locations. This system centralizes all records using a MySQL database named "RMS".
- **Administrative Security**: Without a structured interface, database records are vulnerable to unauthorized changes. This project implements a password-protected CLI to ensure only authorized administrators can modify core train and passenger data.

Objectives of the Project

- **Database Automation**: To programmatically create and maintain relational tables for Users, Trains, Passengers, Bookings, and Cancellations.
- **Dual-Interface Development**: To provide a user-friendly Graphical User Interface (GUI) for general navigation and a Command Line Interface (CLI) for rapid administrative tasks.
- **Streamlined Ticketing**: To automate the booking process by generating and storing PNR numbers, journey details, and fare information.
- **Efficient Record Management**: To enable administrators to perform CRUD (Create, Read, Update, Delete) operations on train schedules and passenger lists through a dedicated CLI.
- **Automated Exporting**: To allow users to generate formatted reports of train and passenger data in `.txt` format at the click of a button.

Scope of the Project

The scope of the RMS project encompasses the functional and administrative requirements of a modern railway hub:

- **Authentication & Security**: Includes user registration and login functionality within the GUI. The Admin CLI is further secured by a password ("admin123").
- **Train & Passenger Management**: The system handles detailed scheduling (Train ID, Name, Source, Destination, Day) and passenger demographics (ID, Name, Age, Gender, Phone).
- **Transaction Processing**: Full support for booking tickets and processing cancellations, including automatic deletion of booking records once a ticket is cancelled and moved to the cancellation log.
- **Reporting & Archiving**: The system can fetch live data from the `TRAIN` and `PASSENGER` tables and export it into readable files like `table_data_train.txt` and `table_data_pasenger.txt`.

3. (SDLC)

3.1 Design

During the design phase, the focus was on creating a logical structure for data and a seamless navigation flow between the GUI and CLI components.

- **Database Design:** A relational schema was designed to minimize redundancy. The system uses primary keys (like `TrainID` and `PNR_No`) and foreign keys to link bookings to specific trains and passengers.

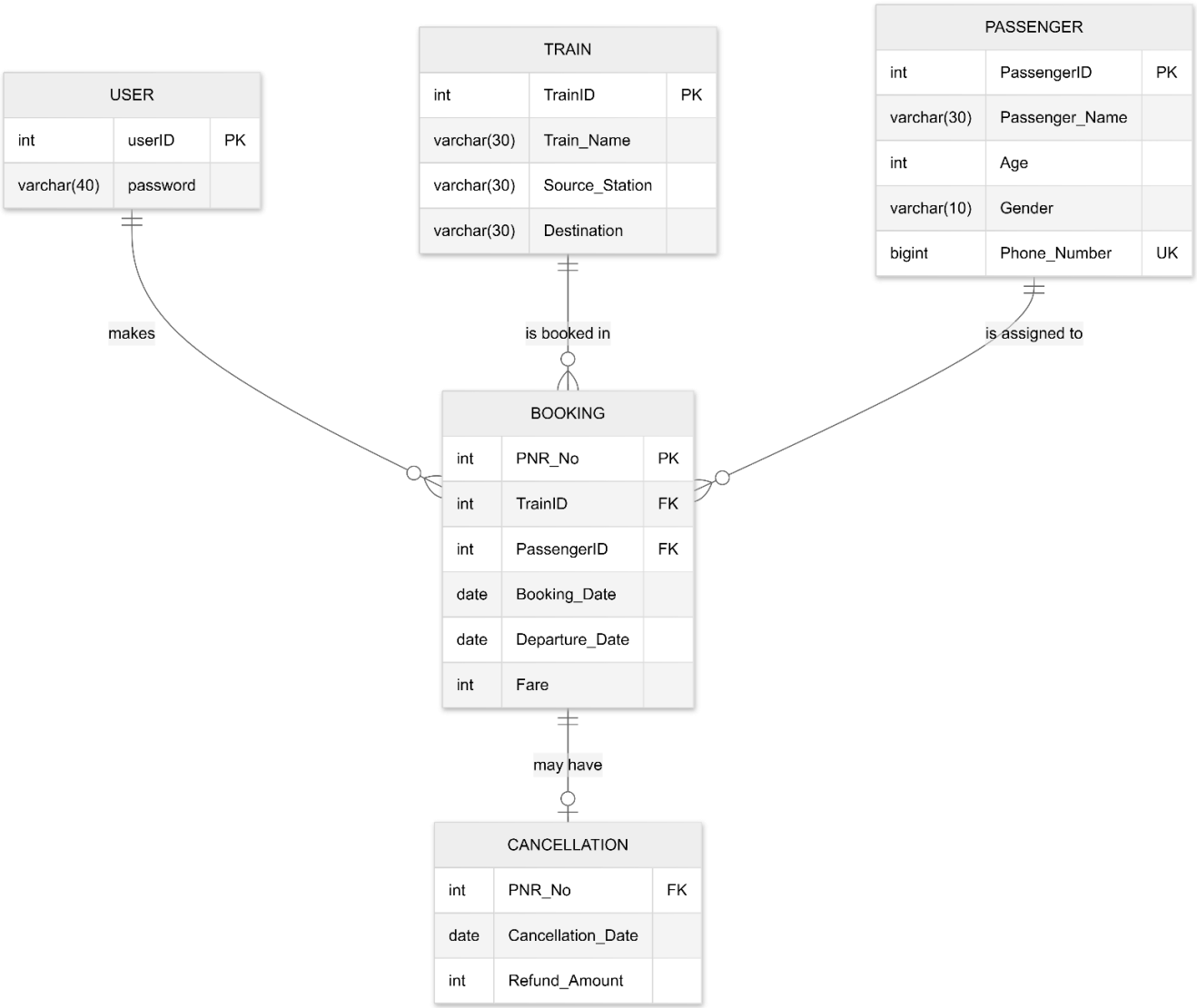

Figure 1: ER Diagram

- **Architectural Design:** The system was split into a **Frontend** (Tkinter GUI), a **Logical Layer** (Modular Python files for business logic), and a **Backend** (MySQL Database).

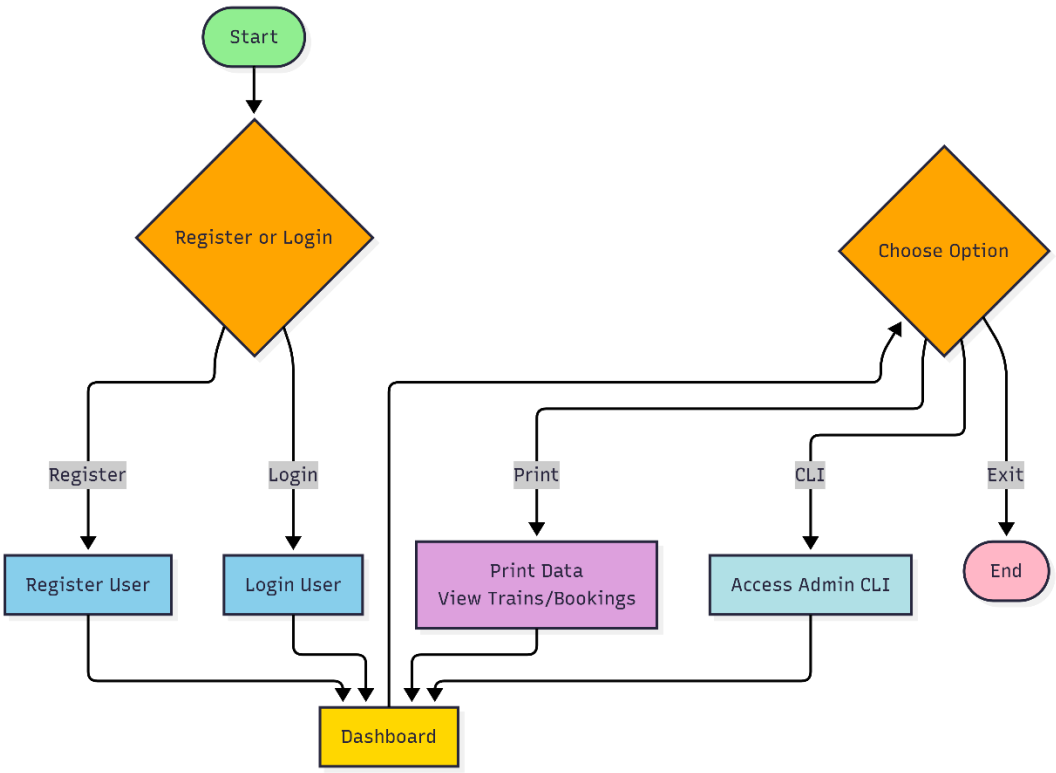

Figure 2: System Architecture Flowchart

3.2 Implementation

The implementation phase involved translating the design into executable Python code using a modular approach.

- **Database Connectivity (`db.py`):** Established the connection string using `mysql.connector` with parameters for host, user, and password.
- **Table Setup (`tables.py`):** Created automated scripts to generate the `RMS` database and all required tables if they do not already exist, ensuring the environment is ready upon first launch.
- **Administrative Logic (`admin_cli.py`):** Developed a text-based interface for heavy data management. This module imports functions from `train.py`, `passengers.py`, and `bookings.py` to allow administrators to add or modify records directly.
- **Export Logic (`printer.py`):** Implemented a reporting feature that uses Python's file handling (`open()`, `write()`) to format database rows into a aligned text table and launch it automatically in the system's default text editor.

3.3 Testing

Testing was conducted iteratively to ensure data integrity and a smooth user experience.

- **Unit Testing:** Individual functions like `add_train()` and `cancel_ticket()` were tested for edge cases (e.g., entering a non-existent PNR).

- **Validation Testing:** The system was checked to ensure that optional fields (like Age or Phone Number) are correctly handled as `NULL` in the database rather than causing a crash.
- **Security Testing:** Verified that the Admin CLI remains inaccessible without the correct password ("admin123").

```
============================
 RAILWAY MANAGEMENT SYSTEM
        ADMIN CLI
============================
Enter admin password:
Access denied ❌
Enter admin password:
Access granted ✅
Type 'help' to see commands
```

Figure 3: Screenshot - Admin CLI Authentication

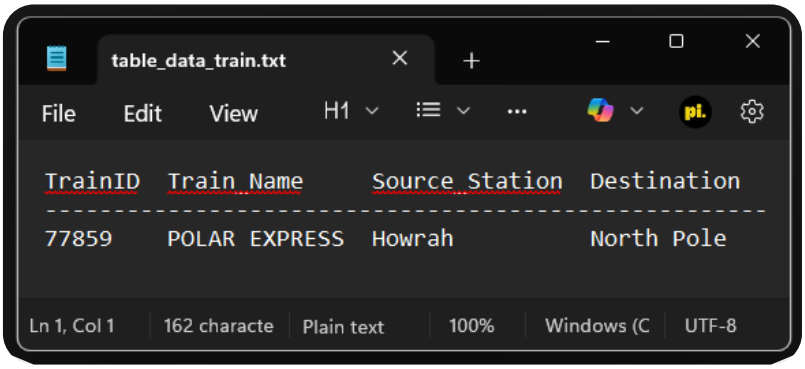

Figure 4: Screenshot - Successful Data Export

3.4 Deployment & Maintenance

The system is deployed locally. For maintenance, the `db.py` file allows for easy modification of database credentials if the system is moved to a different server. The modular structure of the code allows for future updates (like adding a "Seat Availability" feature) without rewriting the core GUI logic.

4. System Design

4.1 Flowchart / Workflow

The system operates on a multi-layered workflow where the user interacts with the GUI for high-level tasks and the Admin CLI for data-intensive management.

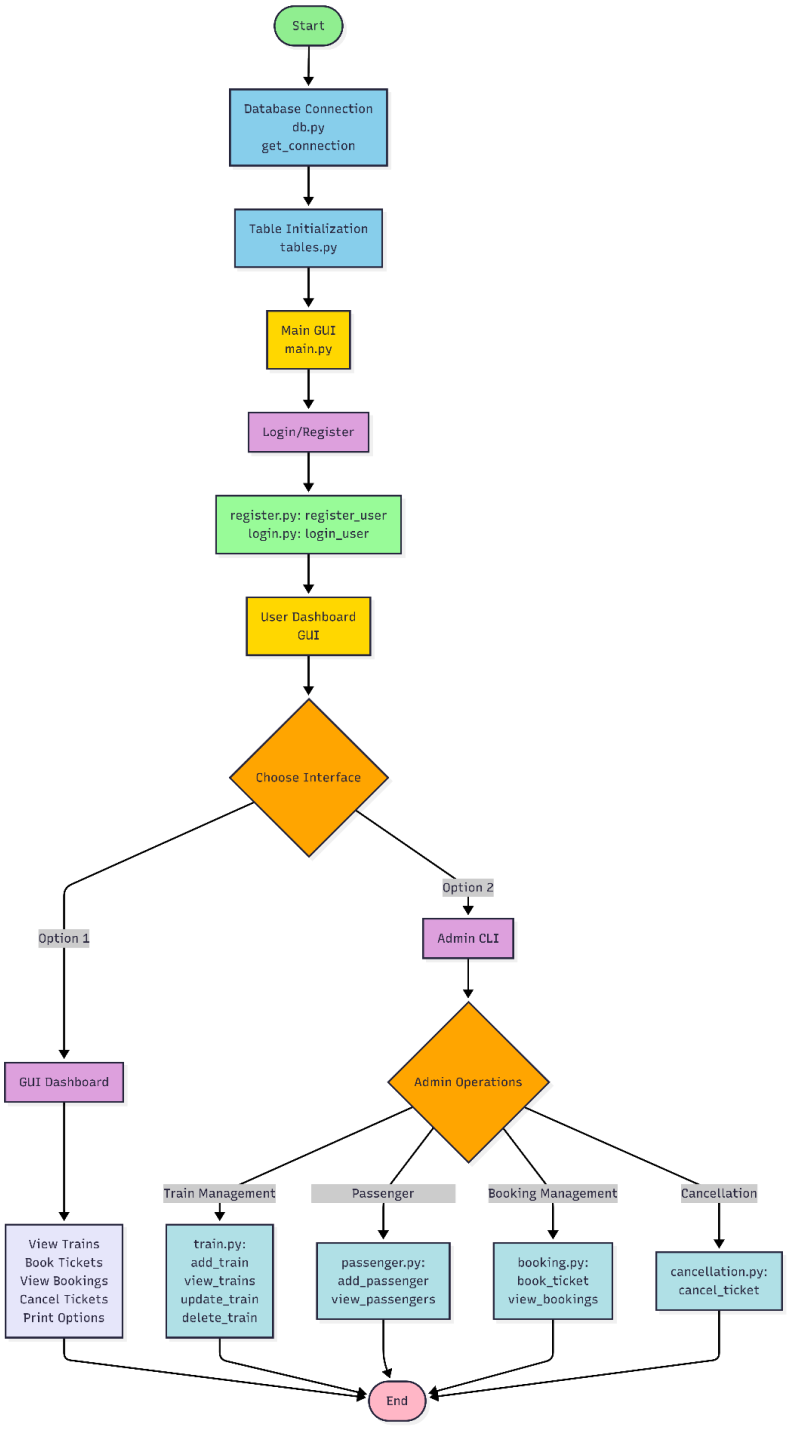

Figure 5: Main Application Workflow

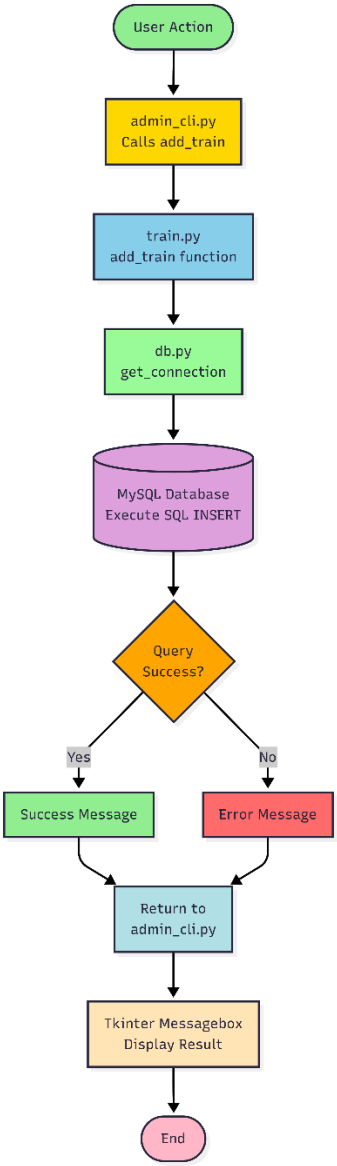

Figure 6: Internal Data Flow

4.2 Module Description

The project is divided into several Python modules, each with a specific responsibility:

- **`main.py`**: The "Brain" of the GUI. It manages the Tkinter window, handles frame switching (Welcome, Login, Dashboard), and initializes the database tables upon startup.
- **`admin_cli.py`**: Provides a robust command-line environment for administrators. It supports nested menus for Trains, Passengers, Bookings, and Cancellations, using keywords or numeric shortcuts.
- **`db.py`**: The database connector. It abstracts the connection details (host, user, password) and provides the `get_connection()` function used by all other modules.
- **`tables.py`**: A setup utility. It contains SQL `CREATE TABLE` statements for all five entities, ensuring that the database schema is consistent across different machines.
- **`train.py`**: Dedicated to the `Train` table. It contains logic for adding new schedules, updating existing routes, deleting trains, and viewing the full schedule.
- **`passengers.py`**: Manages the `Passenger` table. It handles the registration of passenger demographics and ensures data types (like Phone Number as a BigInt) are handled correctly.

- **`bookings.py&cancellation.py`**: These modules handle the transactional logic. `bookings.py` creates the link between trains and passengers, while `cancellation.py` calculates refunds and moves records from the active booking list to the cancellation log.
- **`printer.py`**: The reporting engine. It fetches raw SQL data and uses string formatting to create aligned text reports, which are then automatically opened in the system's default text editor (Notepad).

4.3 Database Design Overview

The system uses a relational database model to ensure data integrity and minimize redundancy.

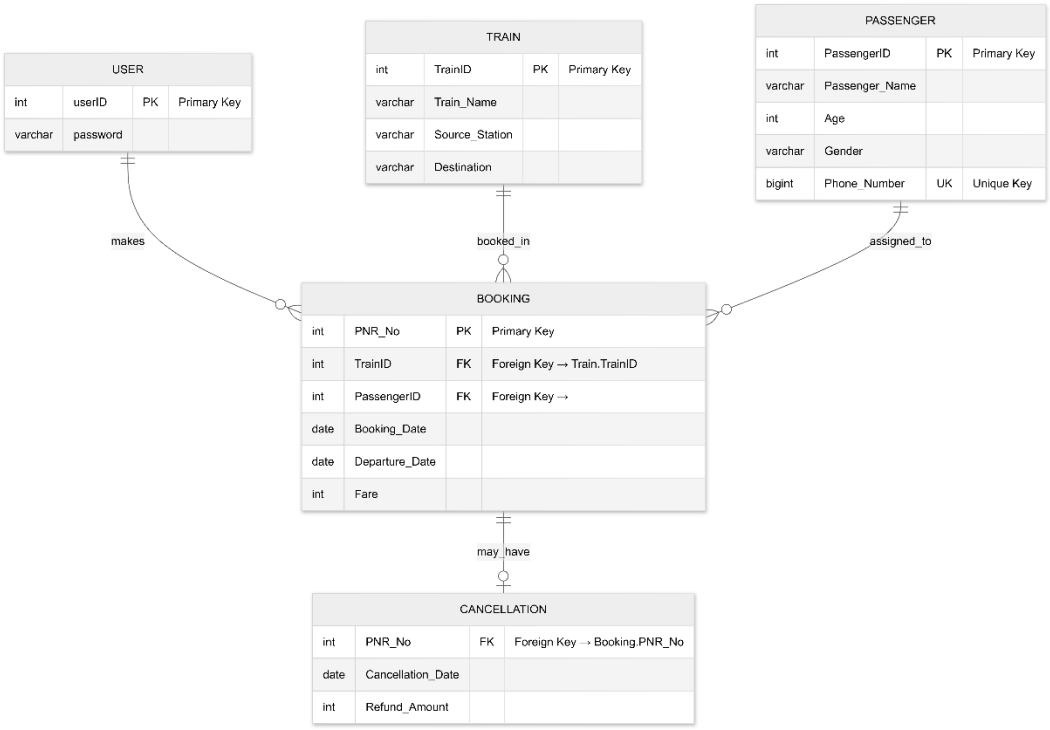

Figure 7: Database Schema Diagram

- **Integrity Rules**:
  - **Cascading Logic**: While the code manually handles deletion (e.g., removing a booking when a ticket is cancelled), the database design enforces that a PNR cannot be cancelled unless it first exists in the Booking table.
  - **Data Constraints**: `Fare` and `Age` are protected by `CHECK` constraints (e.g., `Fare >= 0`) to prevent logical errors in financial data.
  - **Unique Identifiers**: Fields like `Phone_Number` and `UserID` are marked as `UNIQUE` or `PRIMARY KEY` to prevent duplicate account creation.

5. User Interface Design

5.1 GUI Design (Graphical User Interface)

The GUI is designed to be visually engaging and intuitive, using the **Tkinter** library. It follows a "Single Window, Multiple Frames" architecture where the application switches between different views without opening new windows.

- **Design Language**: The interface uses high-quality background images (via `tk.Canvas`) and custom-designed image buttons to create a modern look compared to standard grey UI elements.
- **Key Components**:
  - **Welcome Page**: The entry point featuring the project title and navigation buttons for new and returning users.
  - **Authentication (Register/Login)**: Secure frames where users enter credentials. These pages interact with the `user` table in MySQL.
  - **Dashboard Page**: The central hub for logged-in users. It features a layout with functional icons for:
    - **Exporting Data**: Buttons to trigger the text-based reporting system.
    - **Administrative Access**: A special button to launch the Admin CLI.
    - **Logout/Exit**: Navigation back to the home screen.

Figure 8: Dashboard Page

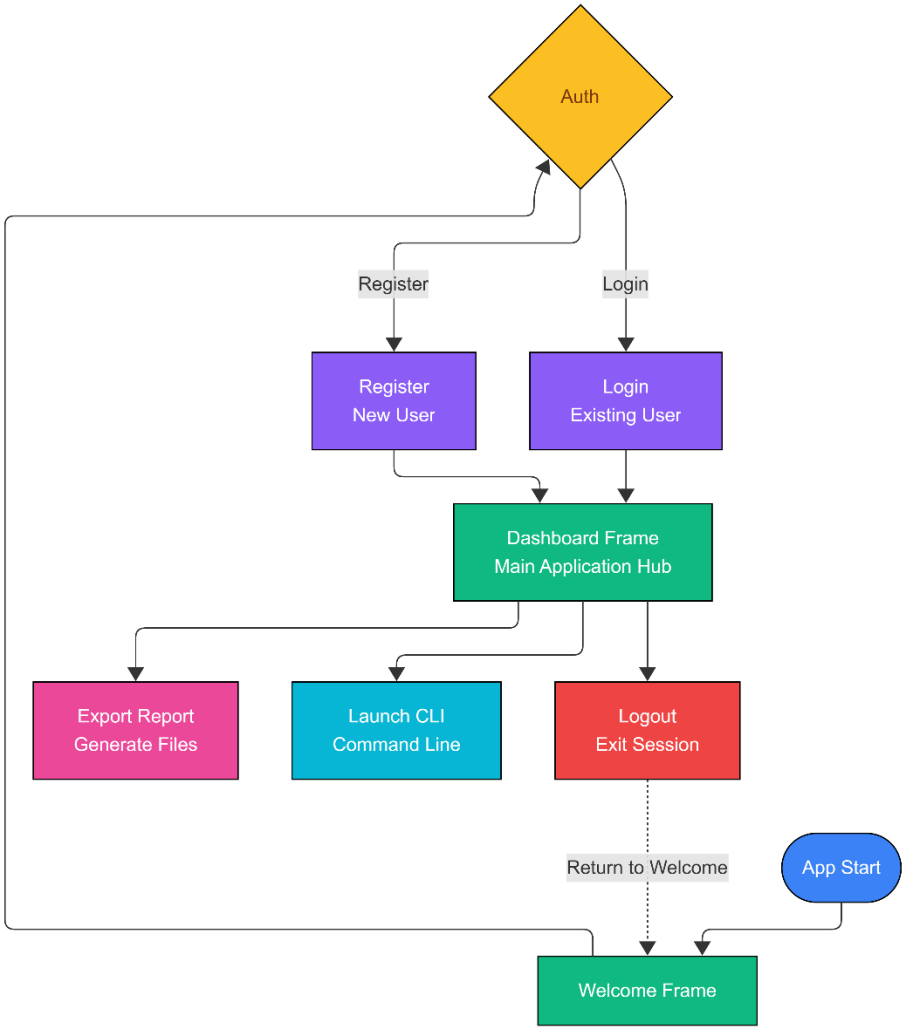

Figure 9: GUI Navigation Flowchart

5.2 CLI Design (Command Line Interface)

The **Admin CLI** (`admin_cli.py`) is a powerful, text-based environment intended for rapid data entry and management by station administrators.

- **Security Layer**: Upon launching, the CLI immediately prompts for an admin password using the `getpass` library (which hides the characters as they are typed). Access is only granted if the password matches `"admin123"`.
- **Menu Structure**: The CLI uses a persistent `while True` loop to keep the session active. It features a hierarchical menu system:
    - **Level 1 (Main Menu)**: Choices for Trains, Passengers, Cancellations, or Bookings.
    - **Level 2 (Sub-Menus)**: Specific actions for each category (Add, Update, Delete, View).

- **Command Versatility**: The system is designed to be flexible; it accepts both numeric inputs (e.g., `1`) and keyword commands (e.g., `trains` or `add`) to cater to different user preferences.
- **Feedback Mechanism**: After every operation, the CLI provides text-based feedback (e.g., "Access granted...", "Fetching records...") and requires a "Press Enter to continue" acknowledgment to prevent the screen from clearing before the user can read the results.

```
============================
 RAILWAY MANAGEMENT SYSTEM
        ADMIN CLI
============================
Enter admin password:
Access denied ❌
Enter admin password:
Access granted ☑
Type 'help' to see commands
```

Figure 10: Admin CLI Menu

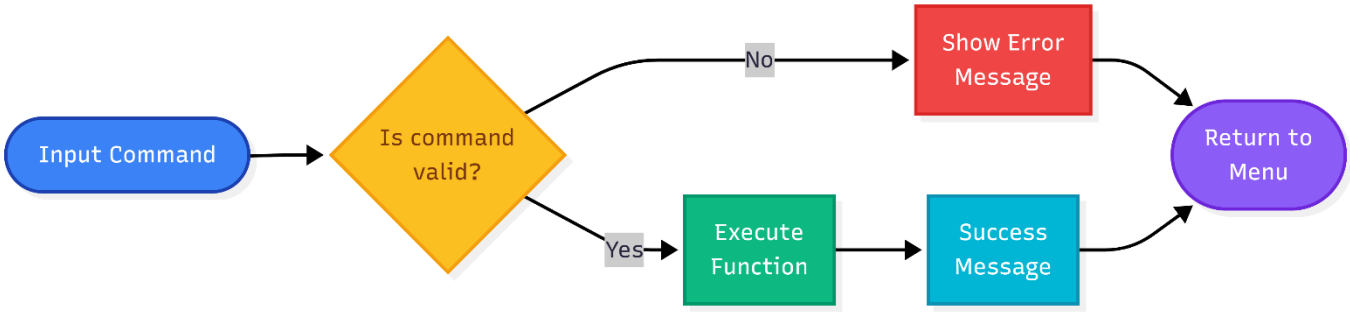

Figure 11: CLI Input Validation Flow

6. Implementation

6.1 Source Code Structure

The project is built using a **Modular Architecture**. Instead of a single monolithic script, the logic is distributed across specialized files. This makes the system easier to debug, maintain, and scale.

- **Entry Point (`main.py`)**: Acts as the orchestrator. it initializes the GUI and handles the transition between different application states (Login, Registration, Dashboard).
- **Database Configuration (`db.py&tables.py`)**: These modules isolate the backend setup. `db.py` handles the physical connection to the MySQL server, while `tables.py` defines the relational schema.
- **Functional Logic Layer**:
  - `train.py`: Logic for scheduling and managing train routes.
  - `passengers.py`: Logic for handling passenger profiles.
  - `bookings.py`: Logic for the ticketing process.
  - `cancellation.py`: Logic for processing refunds and removing records.
- **Reporting Utility (`printer.py`)**: A standalone utility focused on data extraction and file I/O formatting.
- **Administrative Interface (`admin_cli.py`)**: A separate entry point for power users, importing functional logic into a text-based environment.

6.2 Libraries Used

The system leverages a mix of Python's standard library and external packages to provide a rich user experience and persistent data storage.

Pre-defined Libraries

- **`tkinter`**: Used for creating the Graphical User Interface, including windows, canvases, and message boxes.
- **`mysql.connector`**: The driver used to bridge the Python application with the MySQL database.
- **`subprocess`**: Specifically used in `main.py` to launch the `admin_cli.py` script as a separate process from the GUI.
- **`getpass`**: Used in the Admin CLI to allow secure password entry where characters are hidden from the screen.
- **`datetime`**: Used to fetch current timestamps for ticket cancellations.
- **`os`**: Used to interact with the operating system, such as clearing the terminal screen or automatically opening exported text files in Notepad.

User-defined Modules

- **`db`**: Provides `get_connection()` to standardize database access across all files.
- **`tables`**: Contains functions like `create_all_tables()` which are called at every app startup to ensure environment readiness.

- **`train, passengers, bookings, cancellation`**: These modules encapsulate the "Business Logic" of the railway system.
- **`printer`**: Encapsulates the logic for converting SQL result sets into formatted, human-readable text documents

6.3 Source Code

6.3.1 main.py

```
importtkinterastk
importsubprocess
importsys
fromauth.registerimportregister_user
fromauth.loginimportlogin_user
fromprinterimport*
fromdbimportcreate_RMS
fromtablesimportcreate_all_tables
classApp(tk.Tk):
    def__init__(self):
        super().__init__()

        self.title("Railway management system")
        self.geometry("390x844")
        self.resizable(False,False)
        container=tk.Frame(self)
        container.pack(fill="both", expand=True)

        self.frames= {}

        forPagein (WelcomePage, RegisterPage, LoginPage, DashboardPage):
            frame=Page(container, self)
            self.frames[Page] =frame
            frame.grid(row=0, column=0, sticky="nsew")

        self.show_frame(WelcomePage)

    defshow_frame(self, page):
        self.frames[page].tkraise()

classWelcomePage(tk.Frame):
    def__init__(self, parent, controller):
        super().__init__(parent)

        create_all_tables()


        self.canvas=tk.Canvas(self, width=390, height=844)
        self.canvas.pack(fill="both", expand=True)

        self.bg_image=tk.PhotoImage(file="assets/Welcome/bg.png")
        self.register_img=tk.PhotoImage(file="assets/Welcome/btn_register.png")
```

```
        self.login_img=tk.PhotoImage(file="assets/Welcome/btn_login.png")
        self.hero_img=tk.PhotoImage(file="assets/Welcome/Hero.png")

        self.canvas.create_image(195, 422, image=self.bg_image)
        self.canvas.create_image(205, 535, image=self.hero_img)

        # Register button (image only)
        reg_id=self.canvas.create_image(140, 730, image=self.register_img)
                        self.canvas.tag_bind(reg_id,      "<Button-1>",      lambdaevent:
controller.show_frame(RegisterPage))

        # Login button (image only)
        log_id=self.canvas.create_image(320, 730, image=self.login_img)
                        self.canvas.tag_bind(log_id,      "<Button-1>",      lambdaevent:
 controller.show_frame(LoginPage))

classRegisterPage(tk.Frame):
    def__init__(self, parent, controller):
        super().__init__(parent)
        self.controller=controller  # Store the controller instance

        self.canvas=tk.Canvas(self, width=390, height=844)
        self.canvas.pack(fill="both", expand=True)

        self.bg_image=tk.PhotoImage(file="assets/register/bg.png")
        self.register_img=tk.PhotoImage(file="assets/register/register_btn.png")
        self.hero_img=tk.PhotoImage(file="assets/register/Hero.png")
        self.back_img=tk.PhotoImage(file="assets/register/back_btn.png")
        self.ticket_img=tk.PhotoImage(file="assets/register/ticket.png")

        self.canvas.create_image(195, 422, image=self.bg_image)
        self.canvas.create_image(210, 63, image=self.hero_img)
        self.canvas.create_image(195, 500, image=self.ticket_img)

        # Adding input fields
             self.username_entry=tk.Entry(self,  font=("Kumbh  Sans",  12),  fg="black",
bg="#CFC5B9", relief="flat")
        self.username_entry.place(x=60, y=335, width=200, height=30)

          self.password_entry=tk.Entry(self, font=("Arial", 12), fg="black", bg="#CFC5B9",
relief="flat", show="*")
        self.password_entry.place(x=60, y=415, width=200, height=30)

         self.password_reentry=tk.Entry(self, font=("Arial", 12), fg="black", bg="#CFC5B9",
relief="flat")
        self.password_reentry.place(x=60, y=495, width=200, height=30)

        # Register button (image only)
        reg_id=self.canvas.create_image(195, 590, image=self.register_img)
        self.canvas.tag_bind(reg_id, "<Button-1>", self.open_register)
```

```
        # Back button (image only)
        back_id=self.canvas.create_image(40, 63, image=self.back_img)
                            self.canvas.tag_bind(back_id,      "<Button-1>",      lambdaevent:
controller.show_frame(WelcomePage))

    defopen_register(self, event=None):
        whileTrue:
            user_id=self.username_entry.get()
            password=self.password_entry.get()
            reentered_password=self.password_reentry.get()

            ifpassword!=reentered_password:
                tk.messagebox.showwarning("Error", "Passwords do not match!")
                return

            ifnotuser_id.isdigit():
                tk.messagebox.showwarning("Error", "User ID must be a number!")
                return

            try:
                register_user(int(user_id), password)
                tk.messagebox.showinfo("Redirecting", "Registration Successful! Redirecting
to Login Page.")
                break
            exceptExceptionase:
                 tk.messagebox.showwarning("Error", f"Registration failed: {e}. Please try
again.")

        self.username_entry.delete(0, tk.END)
        self.password_entry.delete(0, tk.END)
        self.password_reentry.delete(0, tk.END)
        self.controller.show_frame(LoginPage)  # Navigate to LoginPage

classLoginPage(tk.Frame):
    def__init__(self, parent, controller):
        super().__init__(parent)
        self.controller=controller  # Store the controller instance

        self.canvas=tk.Canvas(self, width=390, height=844)
        self.canvas.pack(fill="both", expand=True)

        # Load images
        self.bg_image=tk.PhotoImage(file="assets/login/bg.png")
        self.login_img=tk.PhotoImage(file="assets/login/login_btn.png")
        self.hero_img=tk.PhotoImage(file="assets/login/Hero.png")
        self.back_img=tk.PhotoImage(file="assets/login/back_btn.png")
        self.ticket_img=tk.PhotoImage(file="assets/login/ticket.png")

        # Add images to canvas
        self.canvas.create_image(195, 422, image=self.bg_image)
        self.canvas.create_image(230, 83, image=self.hero_img)
```

```
        self.canvas.create_image(195, 500, image=self.ticket_img)

        # Adding input fields
              self.username_entry=tk.Entry(self,  font=("Kumbh  Sans",  12),  fg="black",
bg="#ED683D", relief="flat")
        self.username_entry.place(x=60, y=360, width=200, height=30)

          self.password_entry=tk.Entry(self, font=("Arial", 12), fg="black", bg="#ED683D",
relief="flat", show="*")
        self.password_entry.place(x=60, y=445, width=200, height=30)

        # Buttons
        reg_id=self.canvas.create_image(195, 550, image=self.login_img)
        self.canvas.tag_bind(reg_id, "<Button-1>", self.open_login)

        back_id=self.canvas.create_image(40, 70, image=self.back_img)
        self.canvas.tag_bind(back_id, "<Button-1>", self.back)

    defopen_login(self, event=None):
        whileTrue:
            user_id=self.username_entry.get()
            password=self.password_entry.get()

            ifnotuser_id.isdigit():
                tk.messagebox.showwarning("Error", "User ID must be a number!")
                return

            iflogin_user(int(user_id), password):
                  tk.messagebox.showinfo("redirecting", "Login Successful! Redirecting to
Dashboard.")
                self.username_entry.delete(0, tk.END)
                self.password_entry.delete(0, tk.END)
                self.controller.show_frame(DashboardPage)  # Navigate to DashboardPage
                break
            else:
                tk.messagebox.showerror("Error", "Invalid credentials. Please try again.")
                self.username_entry.delete(0, tk.END)
                self.password_entry.delete(0, tk.END)

    defback(self, event=None):
        print("Back to Welcome")
        self.controller.show_frame(WelcomePage)  # Use self.controller to navigate

classDashboardPage(tk.Frame):
    def__init__(self, parent, controller):
        super().__init__(parent)
        self.canvas=tk.Canvas(self, width=390, height=844)
        self.canvas.pack(fill="both", expand=True)

        # Load images
        self.bg_image=tk.PhotoImage(file="assets/dash/bg.png")
```

```
        self.access_img=tk.PhotoImage(file="assets/dash/access_btn.png")
        self.hero_img=tk.PhotoImage(file="assets/dash/Hero.png")
        self.button1_img=tk.PhotoImage(file="assets/dash/1_btn.png")
        self.button2_img=tk.PhotoImage(file="assets/dash/2_btn.png")
        self.button3_img=tk.PhotoImage(file="assets/dash/3_btn.png")
        self.cross_img=tk.PhotoImage(file="assets/dash/cross_btn.png")
        self.ticket_img=tk.PhotoImage(file="assets/dash/ticket.png")
        self.acc_img=tk.PhotoImage(file="assets/dash/acc.png")

        # Add images to canvas
        self.canvas.create_image(195, 422, image=self.bg_image)
        self.canvas.create_image(155, 150, image=self.hero_img)
        self.canvas.create_image(195, 450, image=self.ticket_img)
        self.canvas.create_image(100, 60, image=self.acc_img)

        # Buttons
        access_id=self.canvas.create_image(195, 700, image=self.access_img)
        self.canvas.tag_bind(access_id, "<Button-1>", self.open_cli)

        cross_id=self.canvas.create_image(65, 697, image=self.cross_img)
                        self.canvas.tag_bind(cross_id,     "<Button-1>",     lambdaevent:
controller.show_frame(WelcomePage))

        btn1_id=self.canvas.create_image(340, 325, image=self.button1_img)
        self.canvas.tag_bind(btn1_id, "<Button-1>", self.btn1)

        btn2_id=self.canvas.create_image(340, 445, image=self.button2_img)
        self.canvas.tag_bind(btn2_id, "<Button-1>", self.btn2)

        btn3_id=self.canvas.create_image(340, 570, image=self.button3_img)
        self.canvas.tag_bind(btn3_id, "<Button-1>", self.btn3)

    defopen_cli(self, event=None):
        subprocess.Popen(
            ["cmd", "/k", sys.executable, "admin_cli.py"],
            creationflags=subprocess.CREATE_NEW_CONSOLE
        )

    defbtn1(self, event=None):
        print("Print Available Trains")
        export_table_to_txt_train()

    defbtn2(self, event=None):
        print("Print bookings")
        export_table_to_txt_booking()

    defbtn3(self, event=None):
        print("Print passengers")
        export_table_to_txt_passenger()
```

```
if__name__=="__main__":
    app=App()
    app.mainloop()
```

6.3.2 admin_cli.py

```
# RAILWAY ADMIN CLI
fromtrainimport*
frompassengersimport*
frombookingsimport*
fromcancellationimport*
importgetpass

print("="*40)
print(" RAILWAY MANAGEMENT SYSTEM ")
print("        ADMIN CLI")
print("="*40)


# SIMPLE AUTHENTICATION

whileTrue:
    password=getpass.getpass("Enter admin password: ")
    ifpassword=="admin123":
        print("Access granted...")
        break
    else:
        print("Access denied...")


print("Type 'help' to see commands\n")



# CLI LOOP

whileTrue:
    command=input("RMS> ").strip().lower()

    # Accept both numbers and keyword commands at the main prompt
    ifcommandin ("help", "?"):
        print("""
Available Commands:
-------------------
Trains              - 1 or trains
Passengers          - 2 or passengers
Cancellation        - 3 or cancellation
Bookings            - 4 or bookings
Exit                - exit or quit
Clear Screen        - clear or cls
""")

    elifcommandin ("1", "trains", "train"):
        print("redirecting to train details...")

        print("""
Available Commands:
-------------------
Add trains          - 1 or add
Update trains       - 2 or update
Delete trains       - 3 or delete
View trains         - 4 or view
back                - back
""")
        choice=input("RMS Trains> ").strip().lower()
```

```
        ifchoicein ("1", "add", "add trains"):
            print("Adding train...")            add_train()
        elifchoicein ("2", "update", "update trains"):
            print("Updating train...")
            update_train()
        elifchoicein ("3", "delete", "delete trains"):
            print("Deleting train...")
            delete_train()
        elifchoicein ("4", "view", "view trains"):
            print("Fetching train details...")
            view_trains()
        elifchoice=="back":
            continue
        else:
            print("Unknown command. Type 'back' to return")


    elifcommandin ("2", "passengers", "passenger"):
        print("redirecting to passenger details...")
        print("""
Available Commands:
-------------------
Add passengers      - 1 or add
View passengers     - 2 or view
back                - back
""")
        choice=input("RMS Passengers> ").strip().lower()
        ifchoicein ("1", "add", "add passengers"):
            print("Adding passenger...")
            add_passenger()
        elifchoicein ("2", "view", "view passengers"):
            print("Fetching passenger details...")
            view_passengers()
        elifchoice=="back":
            continue
        else:
            print("Unknown command. Type 'back' to return")


    elifcommandin ("3", "cancellation", "cancel", "cancilation"):
        print("redirecting to cancellation details...")
        print("""
Available Commands:
-------------------
View cancellations  - 1 or view
cancel ticket       - 2 or cancel
back                - back
""")
        choice=input("RMS Cancellation> ").strip().lower()
        ifchoicein ("1", "view", "view cancellations"):
            print("Fetching cancellation details...")
            view_cancellaition()
        elifchoicein ("2", "cancel", "cancel ticket"):
            print("Cancelling ticket...")
            cancel_ticket()
        elifchoice=="back":
            continue
        else:
            print("Unknown command. Type 'back' to return")

    elifcommandin ("4", "bookings", "booking", "book", "book ticket"):
        print("redirecting to booking details...")
        print("""
Available Commands:
-------------------
Book ticket         - 1 or book
View bookings       - 2 or view
back                - back
""")
```

```
        choice=input("RMS Bookings> ").strip().lower()
        ifchoicein ("1", "book", "book ticket"):
            print("Adding booking...")
            book_ticket()
        elifchoicein ("2", "view", "view bookings"):
            print("Fetching booking details...")
            view_bookings()
        elifchoice=="back":
            continue
        else:
            print("Unknown command. Type 'back' to return")

    elifcommandin ("clear", "cls", "clear screen"):
        importos
        os.system("cls")

    elifcommandin ("exit", "quit"):
        print("Exiting CLI")
        break

    else:
        print("Unknown command. Type 'help'")
```

6.3.3db.py

```
importmysql.connectorassl

defcreate_RMS():

    con=sl.connect(
        host="localhost",
        user="root",
        passwd="12345"
    )
    cur=con.cursor()
    cur.execute("CREATE DATABASE IF NOT EXISTS RMS")
    con.commit()
    con.close()

defget_connection():
    returnsl.connect(
        host="localhost",
        user="root",
        passwd="12345",
        database="RMS"
    )
```

6.3.4 tables.py

```
fromdbimportget_connection, create_RMS

defcreate_user_table():
    create_RMS()
    con=get_connection()
    cur=con.cursor()
    cur.execute("USE RMS")
    cur.execute(
        """
```

```
        CREATE TABLE IF NOT EXISTS user (
            userID INT PRIMARY KEY,
            password VARCHAR(40) NOT NULL
        )
        """
    )
    con.commit()
    con.close()

def create_train_table():
    create_RMS()
    con=get_connection()
    cur=con.cursor()
    cur.execute(
        """
        CREATE TABLE IF NOT EXISTS Train (
            TrainID INT PRIMARY KEY,
            Train_Name VARCHAR(30) NOT NULL,
            Source_Station VARCHAR(30) NOT NULL,
            Destination VARCHAR(30) NOT NULL,
            Day DATE NOT NULL
        )
        """
    )
    con.commit()
    con.close()

def create_passenger_table():
    create_RMS()
    con=get_connection()
    cur=con.cursor()
    cur.execute(
        """
        CREATE TABLE IF NOT EXISTS Passenger (
            PassengerID INT PRIMARY KEY,
            Passenger_Name VARCHAR(30) NOT NULL,
            Age INT CHECK (Age > 0),
            Gender VARCHAR(10),
            Phone_Number BIGINT UNIQUE
        )
        """
    )
    con.commit()
    con.close()

def create_booking_table():
    create_RMS()
    con=get_connection()
    cur=con.cursor()
    cur.execute(
        """
```

```
        CREATE TABLE IF NOT EXISTS Booking (
            PNR_No INT PRIMARY KEY,
            TrainID INT,
            PassengerID INT,
            Booking_Date DATE NOT NULL,
            Departure_Date DATE NOT NULL,
            Fare INT CHECK (Fare >= 0),
            FOREIGN KEY (TrainID) REFERENCES Train(TrainID),
            FOREIGN KEY (PassengerID) REFERENCES Passenger(PassengerID)
        )
        """
    )
    con.commit()
    con.close()

defcreate_cancellation_table():
    create_RMS()
    con=get_connection()
    cur=con.cursor()
    cur.execute(
        """
        CREATE TABLE IF NOT EXISTS Cancellation (
            PNR_No INT,
            Cancellation_Date DATE NOT NULL,
            Refund_Amount INT CHECK (Refund_Amount >= 0),
            FOREIGN KEY (PNR_No) REFERENCES Booking(PNR_No)
        )
        """
    )
    con.commit()
    con.close()

defcreate_all_tables():

    create_RMS()
    create_user_table()
    create_train_table()
    create_passenger_table()
    create_booking_table()
    create_cancellation_table()
```

6.3.5 train.py

```
importsys
sys.path.append("c:/Users/USER/Documents/Pytho/Railway-Management-system")
fromdbimportget_connection

importtkinterastk
fromtkinterimportmessagebox


defadd_train():
```

```
    root=tk.Tk()
    root.withdraw()

    try:
        con=get_connection()
        print("Database connection established.")
        cur=con.cursor()

        print("ADD TRAIN DETAILS")

        try:
            trainID=int(input("Enter Train Number: "))
            train_name=input("Enter Train Name: ")
            source_station=input("Enter Source Station: ")
            destination=input("Enter Destination Station: ")

            cur.execute(
                "INSERT INTO Train VALUES (%s, %s, %s, %s)",
                (trainID, train_name, source_station, destination)
            )
            con.commit()

            print("Train Added Successfully")
            messagebox.showinfo("Success!", "Train Added Successfully")

        exceptExceptionase:
            print(f"Error adding train: {e}")
            messagebox.showerror("Error", f"Error adding train: {e}")

    exceptExceptionase:
        print(f"Failed to connect to the database: {e}")
        messagebox.showerror("Database Error", f"Could not connect to the database: {e}")

    finally:
        input("Press Enter to continue...")
        try:
            con.close()
            print("Database connection closed.")
        except:
            pass

defview_trains():
    root=tk.Tk()
    root.withdraw()

    try:
        con=get_connection()
        print("Database connection established.")
        cur=con.cursor()

        print("AVAILABLE TRAINS")
```

```
        cur.execute("SELECT * FROM Train")
        records=cur.fetchall()

        ifnotrecords:
            print("No train records found.")
            messagebox.showinfo("Info", "No Train Records found")
        else:
            print("\nTrain No | Train Name | Source | Destination")
            print("-"*50)
            forrowinrecords:
                print(f"{row[0]:8} | {row[1]:10} | {row[2]:6} | {row[3]}")

    exceptExceptionase:
        print(f"Error fetching train records: {e}")
        messagebox.showerror("Error", f"Error fetching train records: {e}")

    finally:
        input("\nPress Enter to continue...")
        try:
            con.close()
            print("Database connection closed.")
        except:
            pass

defupdate_train():
    root=tk.Tk()
    root.withdraw()

    try:
        con=get_connection()
        print("Database connection established.")
        cur=con.cursor()

        print("UPDATE TRAIN DETAILS")

        try:
            trainID=int(input("Enter Train Number to Update: "))

            # Check if train exists
            cur.execute(
                "SELECT * FROM Train WHERE trainID = %s",
                (trainID,)
            )
            record=cur.fetchone()

            ifnotrecord:
                print("Train Not Found")
                messagebox.showwarning("Invalid Input", "Train Not found")
                input("Press Enter to continue...")
                return
```

```
            # New details
            train_name=input("Enter New Train Name: ")
            source_station=input("Enter New Source Station: ")
            destination=input("Enter New Destination Station: ")

            cur.execute(
                """
                UPDATE Train
                SET train_name = %s, source_station = %s, destination = %s
                WHERE trainID = %s
                """,
                (train_name, source_station, destination, trainID)
            )

            con.commit()
            print("Train Updated Successfully")
            messagebox.showinfo("Success", "Train Updated successfully")

        exceptValueError:
            print("Invalid Train Number")
            messagebox.showwarning("Error", "Invalid train number")

        exceptExceptionase:
            print(f"Error updating train: {e}")
            messagebox.showerror("Error", f"Error updating train: {e}")

    exceptExceptionase:
        print(f"Failed to connect to the database: {e}")
        messagebox.showerror("Database Error", f"Could not connect to the database: {e}")

    finally:
        input("Press Enter to continue...")
        try:
            con.close()
            print("Database connection closed.")
        except:
            pass

defdelete_train():
    con=get_connection()
    cur=con.cursor()

    print("DELETE TRAIN")

    try:
        trainID=int(input("Enter Train Number to Delete: "))

        # Check if train exists
        cur.execute(
            "SELECT * FROM Train WHERE trainID = %s",
```

```
            (trainID,)
        )

        ifnotcur.fetchone():
            print("Train Not Found")
            messagebox.showwarning("Error","Train Not Found")
            input("Press Enter to continue...")
            return

        # Delete train
        cur.execute(
            "DELETE FROM Train WHERE trainID = %s",
            (trainID,)
        )
        con.commit()

        print("Train Deleted Successfully")
        messagebox.showinfo("Sucess","Train Deleted Successfully")

    exceptValueError:
        print("Invalid Train Number")
        messagebox.showwarning("Error","Invalid trian Number")
    finally:
        input("Press Enter to continue...")
        con.close()
```

6.3.6 bookings.py

```
importsys
sys.path.append("c:/Users/USER/Documents/Pytho/Railway-Management-system")
fromdbimportget_connection

importtkinterastk
fromtkinterimportmessagebox


defbook_ticket():
    root=tk.Tk()
    root.withdraw()

    con=get_connection()
    cur=con.cursor()

    print("BOOK TICKET")

    try:
        pnr=int(input("Enter PNR Number: "))
        train_id=input("Enter Train ID (leave blank if not available): ")
        passenger_id=input("Enter Passenger ID (leave blank if not available): ")
        booking_date=input("Enter Booking Date (YYYY-MM-DD): ")
        departure_date=input("Enter Departure Date (YYYY-MM-DD): ")
        fare=input("Enter Fare (leave blank if not available): ")
```

```
        train_id=int(train_id) iftrain_idelseNone
        passenger_id=int(passenger_id) ifpassenger_idelseNone
        fare=int(fare) iffareelseNone

        cur.execute(
            """
            INSERT INTO Booking
            (PNR_No, TrainID, PassengerID, Booking_Date, Departure_Date, Fare)
            VALUES (%s, %s, %s, %s, %s, %s)
            """,
            (pnr, train_id, passenger_id, booking_date, departure_date, fare)
        )

        con.commit()
        print("Ticket Booked Successfully")

    exceptExceptionase:
        print(f"Error {e}")

    finally:
        input("Press Enter to continue...")
        con.close()

defview_bookings():
    root=tk.Tk()
    root.withdraw()

    con=get_connection()
    cur=con.cursor()

    print("BOOKING DETAILS")

    cur.execute("SELECT * FROM Booking")
    records=cur.fetchall()

    ifnotrecords:
        print("No booking records found.")
        messagebox.showerror("Error","No booking records found")
    else:
        print("\nPNR | TrainID | PassengerID | Book Date | Depart Date | Fare")
        print("-"*70)
        forrowinrecords:
            print(f"{row[0]:4} | {row[1]} | {row[2]} | {row[3]} | {row[4]} | {row[5]}")

    input("\nPress Enter to continue...")
    con.close()

defcancel_ticket():
    con=get_connection()
    cur=con.cursor()
```

```
print("CANCEL TICKET")

try:
    pnr=int(input("Enter PNR Number: "))
    cancel_date=input("Enter Cancellation Date (YYYY-MM-DD): ")
    refund=input("Enter Refund Amount (leave blank if none): ")

    refund=int(refund) ifrefundelseNone

    # Check if booking exists
    cur.execute(
        "SELECT Fare FROM Booking WHERE PNR_No = %s",
        (pnr,)
    )
    record=cur.fetchone()

    ifnotrecord:
        print("Booking Not Found")
        messagebox.showerror("Error", "Booking Not Found")
        input("Press Enter to continue...")
        return

    # Insert into Cancellation table
    cur.execute(
        """
        INSERT INTO Cancellation
        (PNR_No, Cancellation_Date, Refund_Amount)
        VALUES (%s, %s, %s)
        """,
        (pnr, cancel_date, refund)
    )

    # Delete from Booking table
    cur.execute(
        "DELETE FROM Booking WHERE PNR_No = %s",
        (pnr,)
    )

    con.commit()
    print("Ticket Cancelled Successfully")
    messagebox.showinfo("Success", "Ticket Cancelled Successfully")

exceptExceptionase:
    print("Error during cancellation")
    messagebox.showerror("Error", f"Error during cancellation: {e}")

finally:
    input("Press Enter to continue...")
    con.close()
```

```
if__name__=="__main__":
    book_ticket()
```

6.3.7 cancellation.py

```
importsys
sys.path.append("c:/Users/USER/Documents/Pytho/Railway-Management-system")
fromdbimportget_connection

importtkinterastk
fromtkinterimportmessagebox
fromdatetimeimportdatetime

defcancel_ticket():
    pnr=input("Enter the PNR number of the ticket to cancel: ")
    con=get_connection()
    cur=con.cursor()

    try:
        # Check if the ticket exists in the booking table
        cur.execute("SELECT * FROM Booking WHERE PNR_No = %s", (pnr,))
        ticket=cur.fetchone()

        ifnotticket:
            print("No ticket found with the given PNR.")
            return

        refund_amount=ticket[5] *0.8
        print(refund_amount)

        cancellation_date=datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        print(cancellation_date)

        # Debugging: Print values before inserting into Cancellation table
            print(f"PNR: {pnr}, Cancellation Date: {cancellation_date}, Refund Amount:
{refund_amount}")

        # Add cancellation details to the Cancellation table
        cur.execute(
              "INSERT INTO Cancellation (PNR_No, Cancellation_Date, Refund_Amount) VALUES
(%s, %s, %s)",
            (pnr, cancellation_date, refund_amount),
        )

        # Delete the ticket from the Booking table
        cur.execute("DELETE FROM Booking WHERE PNR_No = %s", (pnr,))

        # Commit the changes
        con.commit()
        print("Ticket cancelled successfully.")

    exceptExceptionase:
```

```
        print("An error occurred:", e)
        con.rollback()

    finally:
        con.close()


defview_cancellaition():
    root=tk.Tk()
    root.withdraw()

    con=get_connection()
    cur=con.cursor()

    print("BOOK TICKET")
    cur.execute("SELECT * FROM Cancellation")
    records=cur.fetchall()

    ifnotrecords:
        print("No booking records found.")
        messagebox.showerror("Error","No booking records found")
    else:
        print("\nPNR | Cancellation Date | Refund amount")
        print("-"*70)
        forrowinrecords:
            print(f"{row[0]:4} | {row[1]} | {row[2]}")

    input("\nPress Enter to continue...")
    con.close()
```

6.3.8 passengers.py

```
importsys
sys.path.append("c:/Users/USER/Documents/Pytho/Railway-Management-system")
fromdbimportget_connection

importtkinterastk
fromtkinterimportmessagebox


defadd_passenger():
    root=tk.Tk()
    root.withdraw()

    con=get_connection()
    cur=con.cursor()

    print("ADD PASSENGER")

    try:
        pid=int(input("Enter Passenger ID: "))
        name=input("Enter Passenger Name: ")
```

```
        age=input("Enter Age (leave blank if not available): ")
        gender=input("Enter Gender (leave blank if not available): ")
        phone=input("Enter Phone Number (leave blank if not available): ")

        # Convert optional fields
        age=int(age) ifageelseNone
        phone=int(phone) ifphoneelseNone

        cur.execute(
            """
            INSERT INTO Passenger
            (PassengerID, Passenger_Name, Age, Gender, Phone_Number)
            VALUES (%s, %s, %s, %s, %s)
            """,
            (pid, name, age, gender, phone)
        )

        con.commit()
        print("Passenger Added Successfully")
        messagebox.showinfo("Sucess","Passenger Added Successfully")
    exceptException:
        print("Error adding passenger")
        messagebox.showwarning("Error","Error adding passenger")
    finally:
        input("Press Enter to continue...")
        con.close()

defview_passengers():
    root=tk.Tk()
    root.withdraw()

    con=get_connection()
    cur=con.cursor()

    print("PASSENGER LIST")

    cur.execute("SELECT * FROM Passenger")
    records=cur.fetchall()

    ifnotrecords:
        print("No passenger records found.")
        messagebox.showwarning("Error","No passenger records found")
    else:
        print("\nID | Name | Age | Gender | Phone")
        print("-"*55)
        forrowinrecords:
            print(f"{row[0]:2} | {row[1]:15} | {row[2]} | {row[3]} | {row[4]}")

    input("\nPress Enter to continue...")
    con.close()
```

6.3.9 printer.py

```
defexport_table_to_txt_passenger():
    importmysql.connector
    importos
    fromdbimportget_connection



    # ---------- DATABASE CONNECTION ----------
    con=get_connection()
    cursor=con.cursor()

    # ---------- FETCH DATA ----------
    cursor.execute("SELECT * FROM PASSENGER")
    rows=cursor.fetchall()
    headers= [col[0] forcolincursor.description]

    cursor.close()
    con.close()

    # ---------- CALCULATE COLUMN WIDTHS ----------
    col_widths= []
    foriinrange(len(headers)):
        max_len=len(headers[i])
        forrowinrows:
            max_len=max(max_len, len(str(row[i])))
        col_widths.append(max_len+2)  # padding

    # ---------- WRITE FORMATTED TEXT ----------
    txt_file="table_data_pasenger.txt"
    withopen(txt_file, "w", encoding="utf-8") asf:

        # Header
        header_line=""
        fori, hinenumerate(headers):
            header_line+=h.ljust(col_widths[i])
        f.write(header_line+"\n")

        # Separator
        f.write("-"*sum(col_widths) +"\n")

        # Rows
        forrowinrows:
            line=""
            fori, iteminenumerate(row):
                line+=str(item).ljust(col_widths[i])
            f.write(line+"\n")

    # ---------- OPEN IN NOTEPAD ----------
    os.system(f'notepad "{txt_file}"')
```

```
defexport_table_to_txt_booking():
    importmysql.connector
    importos
    fromdbimportget_connection



    # ---------- DATABASE CONNECTION ----------
    con=get_connection()
    cursor=con.cursor()

    # ---------- FETCH DATA ----------
    cursor.execute("SELECT * FROM BOOKING")
    rows=cursor.fetchall()
    headers= [col[0] forcolincursor.description]

    cursor.close()
    con.close()

    # ---------- CALCULATE COLUMN WIDTHS ----------
    col_widths= []
    foriinrange(len(headers)):
        max_len=len(headers[i])
        forrowinrows:
            max_len=max(max_len, len(str(row[i])))
        col_widths.append(max_len+2)

    # ---------- WRITE FORMATTED TEXT ----------
    txt_file="table_data_booking.txt"
    withopen(txt_file, "w", encoding="utf-8") asf:

        # Header
        header_line=""
        fori, hinenumerate(headers):
            header_line+=h.ljust(col_widths[i])
        f.write(header_line+"\n")

        # Separator
        f.write("-"*sum(col_widths) +"\n")

        # Rows
        forrowinrows:
            line=""
            fori, iteminenumerate(row):
                line+=str(item).ljust(col_widths[i])
            f.write(line+"\n")

    # ---------- OPEN IN NOTEPAD ----------
    os.system(f'notepad "{txt_file}"')

defexport_table_to_txt_train():
```

```
importmysql.connector
importos
fromdbimportget_connection



# ---------- DATABASE CONNECTION ----------
con=get_connection()
cursor=con.cursor()

# ---------- FETCH DATA ----------
cursor.execute("SELECT * FROM TRAIN")
rows=cursor.fetchall()
headers= [col[0] forcolincursor.description]

cursor.close()
con.close()

# ---------- CALCULATE COLUMN WIDTHS ----------
col_widths= []
foriinrange(len(headers)):
    max_len=len(headers[i])
    forrowinrows:
        max_len=max(max_len, len(str(row[i])))
    col_widths.append(max_len+2)  # padding

# ---------- WRITE FORMATTED TEXT ----------
txt_file="table_data_train.txt"
withopen(txt_file, "w", encoding="utf-8") asf:

    # Header
    header_line=""
    fori, hinenumerate(headers):
        header_line+=h.ljust(col_widths[i])
    f.write(header_line+"\n")

    # Separator
    f.write("-"*sum(col_widths) +"\n")

    # Rows
    forrowinrows:
        line=""
        fori, iteminenumerate(row):
            line+=str(item).ljust(col_widths[i])
        f.write(line+"\n")

# ---------- OPEN IN NOTEPAD ----------
os.system(f'notepad "{txt_file}"')
```

6.3.10 login.py

```
# auth/login.py
```

```
importsys
sys.path.append("c:/Users/USER/Documents/Pytho/Railway-Management-system")
fromdbimportget_connection
importtkinterastk
fromtkinterimportmessagebox


deflogin_user(user_id, password):
    root=tk.Tk()
    root.withdraw()

    try:
        con=get_connection()
        print("Database connection established.")
        cur=con.cursor()

        # Check credentials
        cur.execute(
            "SELECT userID FROM User WHERE userID = %s AND password = %s",
            (user_id, password)
        )

        ifcur.fetchone():
            print("Login Successful!")
            messagebox.showinfo("Success", "Login Successful!")
            returnTrue
        else:
            print("Invalid User ID or Password")
            messagebox.showerror("Error", "Invalid User ID or Password")
            returnFalse

    exceptValueError:
        print("Invalid input! User ID must be numeric.")
        messagebox.showwarning("Warning", "Invalid input! User ID must be numeric.")
        returnFalse

    exceptExceptionase:
        print(f"An error occurred during login: {e}")
        messagebox.showerror("Error", "An unexpected error occurred.")
        returnFalse

    finally:
        try:
            con.close()
            print("Database connection closed.")
        except:
            pass
```

6.3.11 register.py

```
# auth/register.py
```

```
importsys
sys.path.append("c:/Users/USER/Documents/Pytho/Railway-Management-system")
fromdbimportget_connection

importtkinterastk
fromtkinterimportmessagebox

defregister_user(user_id, password):

    root=tk.Tk()
    root.withdraw()

    try:
        con=get_connection()
        print("Database connection established.")
        cur=con.cursor()

        # Check if user already exists
        cur.execute("SELECT * FROM User WHERE userID = %s", (user_id,))
        ifcur.fetchone():
            print("User ID already exists!")
            messagebox.showwarning("Error", "User ID already exists!")
        else:
            # Insert new user
            cur.execute(
                "INSERT INTO User (userID, password) VALUES (%s, %s)",
                (user_id, password)
            )
            con.commit()
            print("Registration Successful!")
            messagebox.showinfo("Success", "Registration Successful!")

    exceptValueError:
        print("Invalid input! User ID must be a number.")
        messagebox.showwarning("Invalid Input", "User ID must be a number.")

    exceptExceptionase:
        print(f"An error occurred during registration: {e}")
        messagebox.showwarning("Error", "An unexpected error occurred.")

    finally:
        try:
            con.close()
            print("Database connection closed.")
        except:
            pass
```

7. Database Design

7.1 Table Structures

The system consists of five interconnected tables. Below are the specific schemas as defined in `tables.py`:

1. Table: **`user`** Stores the credentials for users accessing the GUI.

USER			
int	userID	PK	Primary Key
varchar(40)	password		

2. Table: **`Train`** Maintains the schedule of available trains

TRAIN			
int	TrainID	PK	Primary Key
varchar(30)	Train_Name		
varchar(30)	Source_Station		
varchar(30)	Destination		

3. Table: **`Passenger`** Contains demographic information for passengers.

PASSENGER			
int	PassengerID	PK	Primary Key
varchar(30)	Passenger_Name		
int	Age		
varchar(10)	Gender		
bigint	Phone_Number	UK	Unique Key

4. Table: **`Booking`** The core transactional table linking trains and passengers.

BOOKING			
int	PNR_No	PK	Primary Key
int	TrainID	FK	Foreign Key → Train.TrainID
int	PassengerID	FK	Foreign Key → Passenger.PassengerID
date	Booking_Date		
date	Departure_Date		
int	Fare		

5. Table: `Cancellation` Logs all cancelled tickets and their associated refunds.

CANCELLATION			
int	PNR_No	FK	Foreign Key → Booking.PNR_No
date	Cancellation_Date		
int	Refund_Amount		

7.2 Relationships

The database utilizes a **Star Schema** approach where the `Booking` table acts as the central fact table.

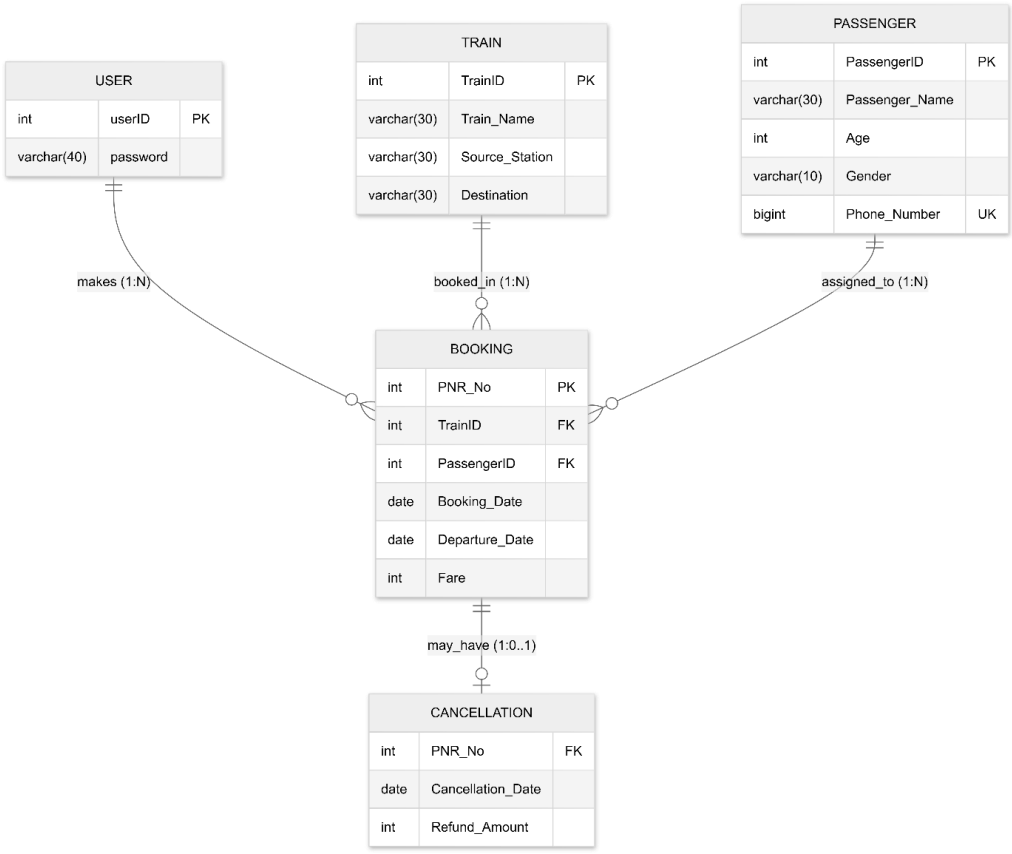

Figure 12: : Database Schema Diagram

- **One-to-Many (`Train` to `Booking`)**: One train can have multiple associated bookings, but a single PNR belongs to only one train.
- **One-to-Many (`Passenger` to `Booking`)**: A single passenger can book multiple tickets over time.
- **One-to-One / Extension (`Booking` to `Cancellation`)**: When a ticket is cancelled, the PNR is used as a reference to calculate the refund, ensuring that only valid existing bookings can be processed for cancellation.

7.3 Sample Records

Below is a representation of how data is populated within the system:

Sample: Train

TrainID (PK)	Train_Name	Source_Station	Destination	Day
12001	Shatabdi Exp	New Delhi	Bhopal	2024-12-25
12952	Rajdhani Exp	Mumbai	New Delhi	2024-12-26

Sample: Booking

PNR_No (PK)	TrainID (FK)	PassengerID (FK)	Booking_Date	Departure_Date	Fare
845512	12001	101	2024-12-01	2024-12-25	1200

Sample: Cancellation

PNR_No (FK)	Cancellation_Date	Refund_Amount
845512	2024-12-10	960

8. Testing

Testing is a critical phase of the SDLC that ensures the Railway Management System is reliable, secure, and free from logical errors. The testing for this project focused on data validation, database integrity, and user authentication.

8.1 Testing Strategy

The project underwent several levels of testing:

- **Unit Testing:** Individual functions in modules like `train.py` and `cancellation.py` were tested to ensure they perform specific tasks (e.g., calculating exactly 80% of the fare for a refund).
- **Integration Testing:** Tested the interaction between the GUI (`main.py`) and the backend logic to ensure that clicking a button correctly triggers a database query.
- **Validation Testing:** Checking if the system handles "bad data" (e.g., entering text into a Fare field) without crashing.

TEST CASES

Test Case ID	Feature Tested	Input/Action	Expected Result	Status
TC-01	Admin Authentication	Enter password "admin123"	Access granted to CLI menu.	Pass
TC-02	Admin Authentication	Enter wrong password	"Access denied" message and exit.	Pass
TC-03	Data Integrity	Book a ticket for a non-existent TrainID	System should throw an Integrity Error (Foreign Key).	Pass
TC-04	Refund Calculation	Cancel a ticket with Fare: 1000	Record deleted; Refund of 800 added to Cancellation table.	Pass
TC-05	Export Logic	Click "Export Train Table"	File table_data_train.txt created and opened in Notepad.	Pass
TC-06	Input Validation	Leave "Passenger Name" blank	Error message: "Please fill all required fields."	Pass

```
============================
 RAILWAY MANAGEMENT SYSTEM
        ADMIN CLI
============================
Enter admin password:
Access denied ❌
Enter admin password:
Access granted ✅
Type 'help' to see commands
```

Figure 12: Screenshot - Admin CLI Authentication

8.3 Error Handling and Robustness

The system is designed to handle common runtime errors gracefully:

1. **Database Connection Errors:** In `db.py`, the connection logic is wrapped in `try-except` blocks. If the MySQL service is not running, the system alerts the user instead of crashing.
2. **Constraint Violations:** The `tables.py` script enforces `CHECK` constraints (e.g., `Age > 0`). If a user attempts to enter an invalid age, the MySQL connector catches the exception and displays a message via Tkinter's `messagebox`.
3. **Missing Records:** In `cancellation.py`, before processing a refund, the code performs a `SELECT` query to verify the PNR exists. If the PNR is invalid, the system notifies the admin that "No such booking found."

8.4 Verification of Output

To verify that the system works as intended, the database state was manually checked after operations:

- After **Booking**: Verified that the row appeared in the `Booking` table via MySQL Workbench.
- After **Cancellation**: Verified that the row was removed from the `Booking` table and a corresponding entry was created in the `Cancellation` table.

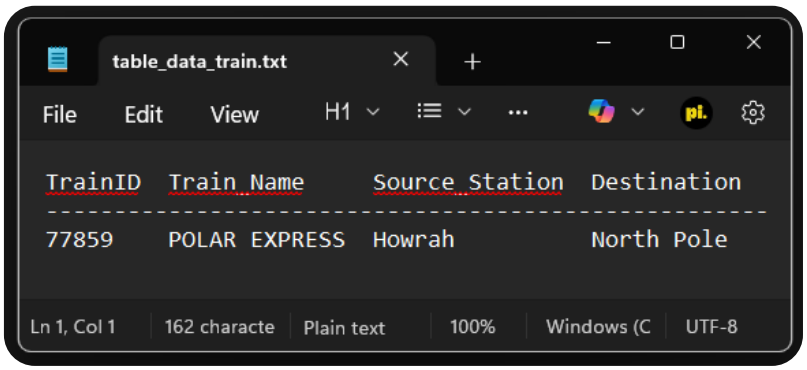

Figure 13: Screenshot - Exported File Layout

9. Demonstration

This section demonstrates the end-to-end functionality of the Railway Management System, showcasing the transition from the Graphical User Interface to the Administrative CLI and the final data outputs.

9.1 GUI Navigation and User Access

When the application is launched, the user is greeted by a high-resolution **Welcome Page**. From here, a user can navigate to the **Registration** frame to create an account or the **Login** frame if they are already registered.

Upon successful authentication, the system transitions to the **Dashboard**.

Figure 14: Screenshot - Login Screen

Figure 15 **Screenshot - User Dashboar.**

9.2 Administrative CLI Operations

The **Admin CLI** is the primary tool for database management. It is launched from the GUI and requires a separate administrative password ("admin123").

- **Train Management**: Admins can view the full schedule or add new trains. The output is displayed in a clean, tabular format within the terminal.
- **Passenger Records**: Admins can register new passengers or search for existing ones using their Passenger ID.
- **Transaction Logs**: The CLI allows admins to view all active bookings and the history of cancellations.

```
============================
 RAILWAY MANAGEMENT SYSTEM
        ADMIN CLI
============================
Enter admin password:
Access denied ❌
Enter admin password:
Access granted ☑️
Type 'help' to see commands
```

Figure 13: Screenshot - Admin CLI Login

```
redirecting to train details...

Available Commands:
-------------------
Add trains        - 1 or add
Update trains     - 2 or update
Delete trains     - 3 or delete
View trains       - 4 or view
back              - back
```

Figure 17: Screenshot - Viewing Train Schedule in CLI]

9.3 Booking and Cancellation Workflow

The system processes transactions in real-time.

1. **Booking**: A PNR is generated, and details are saved to the `Booking` table.
2. **Cancellation**: When an admin enters a PNR for cancellation, the system:
    - Retrieves the original fare.
    - Calculates a **80% refund**.
    - Logs the entry in the `Cancellation` table.
    - Deletes the record from the `Booking` table to free up the status.

```
RMS> 3
redirecting to cancellation details...

Available Commands:
-------------------
View cancellations  - 1 or view
cancel ticket       - 2 or cancel
back                - back
```

Figure 18: Screenshot - Cancellation Process

9.4 Data Export (Reporting)

The "Export" feature on the Dashboard triggers the `printer.py` module. It queries the database and generates a text file.

- **Output Format**: The generated file (e.g., `table_data_train.txt`) uses string padding to ensure that columns are perfectly aligned for readability.
- **Automatic Launch**: The system uses the `os.startfile` command to immediately open the report in the default text editor (Notepad) so the user can print or save it.

10. Conclusion

The **Railway Management System** project successfully integrates a Python-based frontend with a MySQL relational database to provide a comprehensive solution for railway operations.

Key Achievements:

- **Efficiency**: Tasks that previously required manual calculation (like refunds) are now handled instantly by the system logic.
- **Data Integrity**: By using primary and foreign keys, the system prevents "orphaned" records (e.g., a booking without a valid train).
- **Accessibility**: The dual-interface approach allows casual users to navigate via a GUI while providing power users (admins) with a fast CLI.
- **Extensibility**: The modular code structure allows for future enhancements, such as an online payment gateway or real-time seat availability tracking.

This project serves as a foundational model for building database-driven applications that require secure administrative control and user-friendly reporting.

11. Bibliography

- *Python 3.x Documentation*: For Tkinter and Subprocess implementation.
- *MySQL Connector/Python Developer Guide*: For database integration and error handling.
- *W3Schools & Stack Overflow*: For SQL query optimization and logic flow debugging.
- *Figma Design*
- *Photoshop*
- Arihant All in One Computer Science with Python Class 12 for CBSE Exams 2025-26
- Sumita Arora computer science with python class 12